

# JavaScript (ES6+) Master Cheat Sheet

Core Concepts, Modern Syntax, Asynchronous Patterns & Memory Management

## 1. Variables, Scoping & Data Types

- **Declaration:** `const` (block-scoped, immutable binding), `let` (block-scoped, re-assignable), `var` (function-scoped, hoisted as `undefined`).
- **Primitives (Stored by Value):** `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint`.
- **Objects (Stored by Reference):** Objects, Arrays, Functions, Dates, Maps, Sets.

```
// Type checking & Nullish Coalescing
const count = data?.items?.length ?? 0; // Optional chaining + Nullish fallback
const isNumber = typeof count === 'number';
```

## 2. Functions, Closures & Context (`this`)

- **Arrow Functions:** Lexically bind `this` from surrounding scope; cannot be used as constructors.
- **Closures:** An inner function retains access to variables declared in its outer lexical scope even after the outer function finishes executing.
- **Explicit Binding:** `call(thisArg, arg1, arg2)`, `apply(thisArg, [args])`, `bind(thisArg)` returns a bound function.

```
// Closure Example (Private Counter)
function createCounter() {
  let count = 0;
  return () => ++count;
}
const increment = createCounter();
```

## 3. Array Operations & Destructuring

```
const users = [{ id: 1, name: 'Alice', active: true }, { id: 2, name: 'Bob', active: false }];

// Imperative vs Declarative Methods
const activeNames = users.filter(u => u.active).map(u => u.name);
const total = [10, 20, 30].reduce((acc, curr) => acc + curr, 0);

// Rest & Spread Operator
const [first, ...rest] = users;
const updatedUser = { ...first, name: 'Updated' };
```

## 4. Asynchronous JavaScript: Promises & Async/Await

- **Event Loop:** Executes synchronous Call Stack → Microtask Queue (Promises, `process.nextTick`) → Macrotask Queue (`setTimeout`, DOM events).

```
// Async/Await with Error Handling
async function fetchData(url) {
  try {
```

```
const res = await fetch(url);
if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
return await res.json();
} catch (err) {
  console.error('Fetch failed:', err.message);
}

// Parallel Requests
const [userRes, postsRes] = await Promise.all([fetch('/user'), fetch('/posts')]);
```

## 5. Prototypes & ES6 Classes

---

```
class Person {
  #privateId; // Private field (ES2022)
  constructor(name, id) {
    this.name = name;
    this.#privateId = id;
  }
  getId() { return this.#privateId; }
}

class Developer extends Person {
  constructor(name, id, tech) {
    super(name, id);
    this.tech = tech;
  }
}
```